Deterministic Simon

Four buttons, four cues, RGB status, and sound — Mega 2560

If seed, configuration, time, and input match, every observable must match.

Lesson	006 — integration project	**Time**	150–210 minutes
Engine	host verified	**Hardware adapter**	experimental
Evidence	replay, fault tests, measurements	**Board**	Arduino Mega 2560
Safety	E1: USB-powered indicators, controls, and passive piezo only		

1 Mission

Build a memory game that plays a sequence of four neutral cues, accepts one debounced button at a time, grows the sequence, and reports success or failure with light and sound. First prove the state machine with a fixed cue source. Then prove the versioned PRNG. Only then connect hardware.

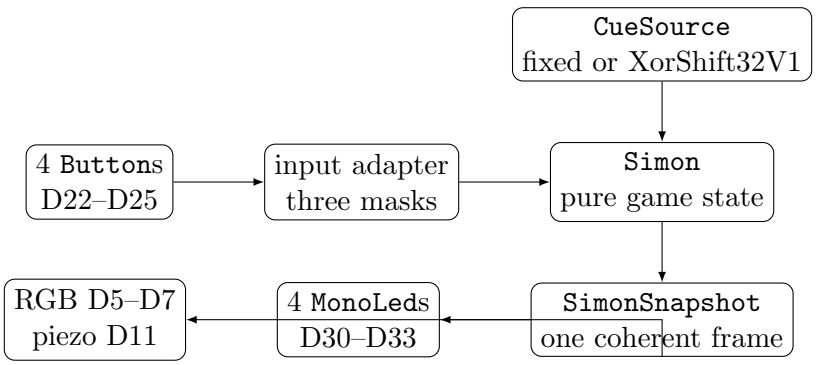
By the end, you can:

- compose four `Buttons`, four `MonoLeds`, one `RgbLed`, and a `PiezoSounder` around `Simon`;
- pack complete input into masks without depending on button update order;
- trace every `SimonPhase`, deadline, and terminal outcome;
- reproduce a game from algorithm version, seed, configuration, timestamps, and input observations;
- reject simultaneous input and injected source failure deterministically;
- calculate LED current and record a Mega 2560 hardware acceptance result.

Safety checkpoint. Disconnect USB and every other power source before changing wiring. Every LED die needs its own 330Ω series resistor: seven resistors total for four cue LEDs plus three RGB channels. Use a passive piezo element, not a speaker or an unidentified active module; place 100Ω in series. Never connect an output to 5 V or GND directly. Stop for heat, odor, board resets, distortion, or unexpected brightness.

Neutral-cue boundary. This project is a memory game. `CueId` values have no launcher, ignition, continuity, or radio meaning. Do not connect the project to a firing system, initiator, transmitter, captured remote, relay, or other energetic load. Later show-cue work remains an inert host/indicator simulation and provides no RF replay or launcher interface.

2 Composition, not inheritance



`Simon` owns no GPIO and reads no clock. The adapter samples all buttons, then presents one immutable `SimonInput`. Output adapters consume one snapshot after the update. `RgbLed` composes three supported `PwmOutputs` and records the three channel/resistor specifications. `PiezoSounder` owns its pin and timer claim. The canonical `examples/Lesson006Simon` sketch uses this complete composition.

3 Orientation drawing

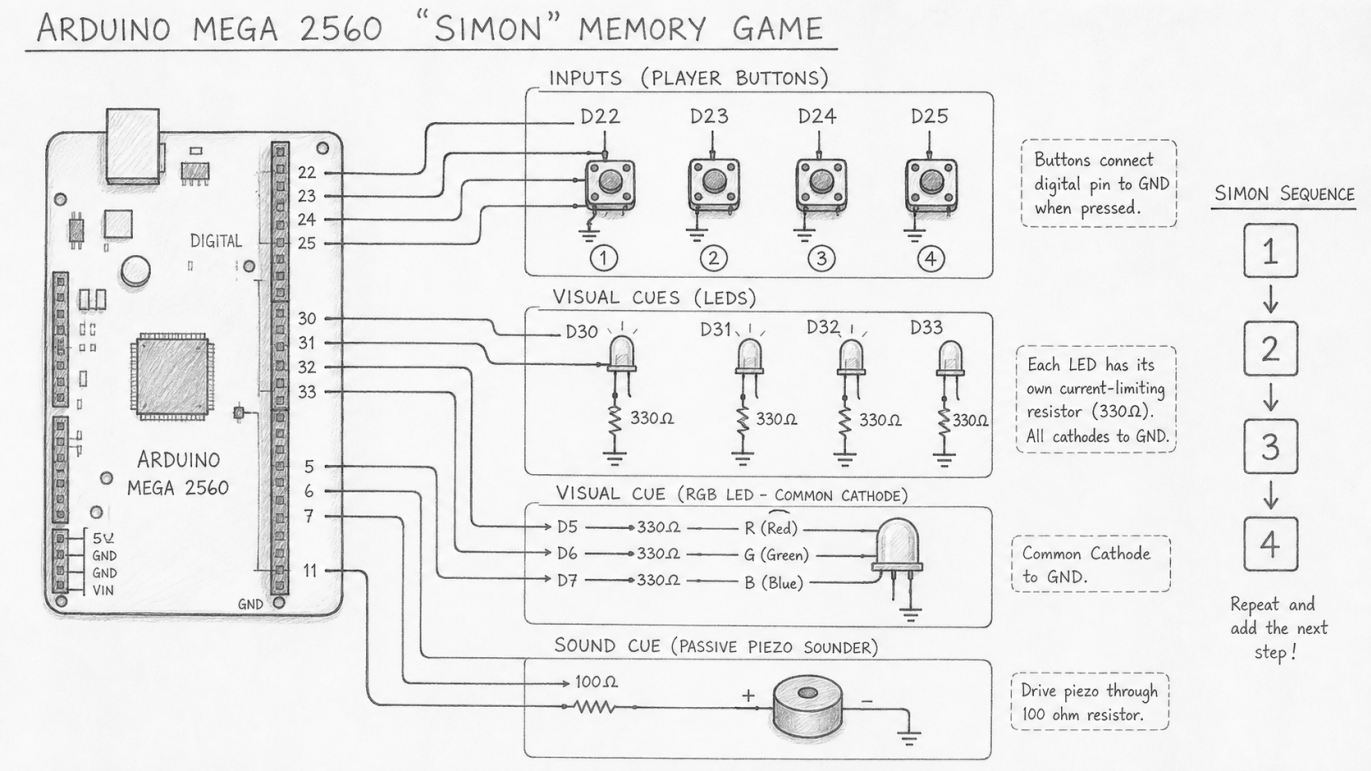
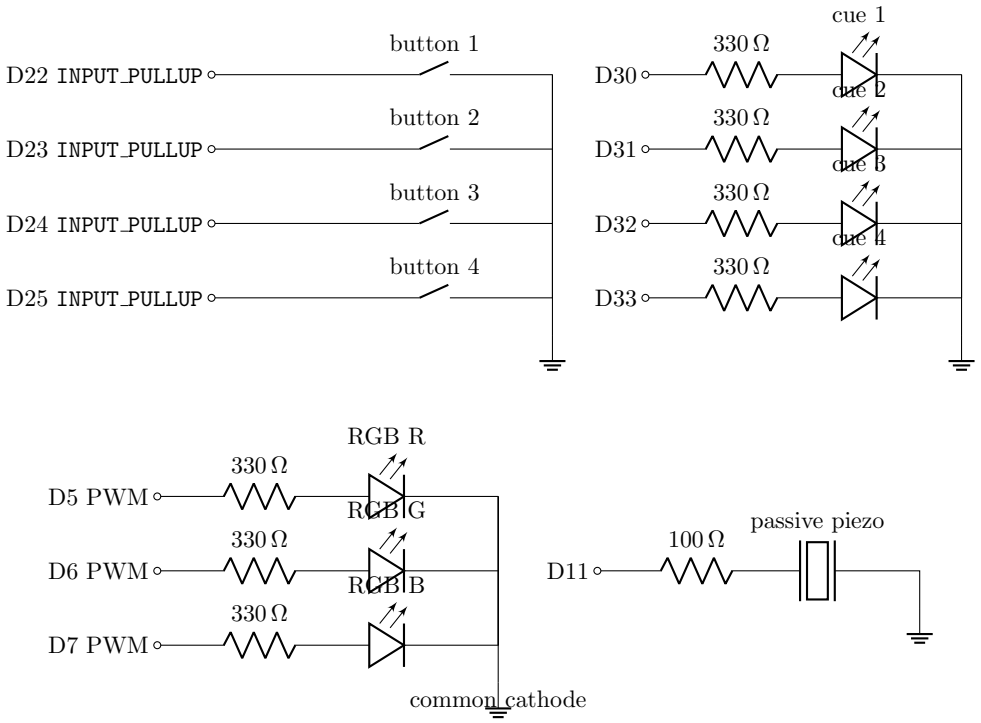


Figure 1: Conceptual pencil orientation for the complete control panel. This drawing helps locate functional groups, but it is **not authoritative wiring guidance**: board-header geometry, breadboard connectivity, component lead order, and polarity may be simplified. Build and verify only from the exact schematics, pin table, part datasheets, and unpowered continuity checks below.

4 Exact electrical circuit

Pin assignment.

Function	Mega pins	Connection	Policy
buttons 1–4	D22–D25	pin → switch → GND	internal pull-up
cue LEDs 1–4	D30–D33	pin → 330 Ω → anode; cathode → GND	active-high
RGB R/G/B	D5/D6/D7	pin → 330 Ω → anode; common cathode → GND	PWM
passive piezo	D11	pin → 100 Ω → piezo +; piezo − → GND	tone



The schematics are literal. Verify RGB package pinout from its datasheet; lead order is not universal. A common-anode RGB part requires different wiring and inverted duty semantics. Four-pin tactile switches often join same-side legs internally; find the contacts that change with a continuity meter while unpowered. All ground symbols join Mega GND.

4.1 Current budget

For each LED channel,

$$I = \frac{V_{OH} - V_f}{330\,\Omega}.$$

With a measured loaded $V_{OH} = 4.8$ V and red $V_f = 2.0$ V, the example is 8.5 mA. Measure or obtain V_f for each actual die. The normal cue design lights one cue LED; the RGB status can light three channels. Calculate the worst commanded simultaneous current, then compare it with every applicable per-pin, port-group, and package limit in the ATmega2560 datasheet:

$$I_{\text{worst}} = I_{\text{cue}} + I_R + I_G + I_B = \_\_\_\_\_ \text{ mA}.$$

Also calculate the wiring-fault/software-fault bound with all seven dies commanded on. Use each die's minimum datasheet V_f, not one typical red value:

$$I_{\text{seven}} = \sum_{n=1}^{7} \frac{V_{OH} - V_{f,\min,n}}{330\,\Omega} = \_\_\_\_\_ \text{ mA}.$$

Do not run the adapter unless the normal and fault bounds leave margin below every pin, port-group, and device operating limit. The piezo's capacitive drive must also satisfy its identified part rating; it is not included in the LED sum.

Do not treat an absolute maximum as a design target. Brightness is not proof of safe current. The passive piezo is primarily capacitive; never substitute an 8Ω speaker driven directly from D11.

5 Engine API and immutable configuration

```
const adk::CueId fixedCues[] =
{
    adk::CueId::One,
    adk::CueId::Three
};

adk::FixedCueSource source (fixedCues, 2);

adk::SimonConfig config;
config.cueOnDuration  = adk::Duration (100);
config.cueGapDuration = adk::Duration (50);
config.inputTimeout   = adk::Duration (500);
config.resultDuration = adk::Duration (100);
config.startingLength = 1;
config.growthPerRound = 1;
config.maximumLength  = 2;

adk::Simon game (config, source);

if (!game.initialize ().ok ())
{
    // Remain electrically inactive; report the status outside the engine.
}
```

The object stores a copy of `SimonConfig` and a non-owning relationship to the cue source, which must outlive it. No heap, exception, RTTI, callback, or hidden clock is involved. The current engine is host verified. The complete four-button/RGB/sound hardware adapter remains experimental until its Mega acceptance record is complete.

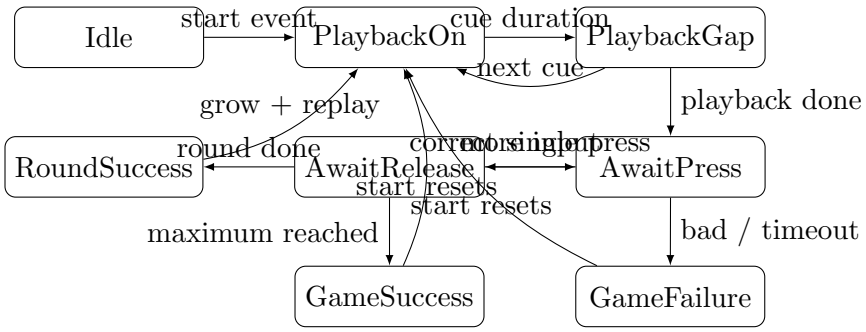
5.1 Input is a complete observation

```
adk::SimonInput input;

for (uint8_t index = 0; index < adk::Simon::cueCount; ++index)
{
    buttons[index]->update (now);
    const uint8_t bit = static_cast<uint8_t> (1u << index);

    if (buttons[index]->pressed ())
    {
        input.activeMask |= bit;
    }

    if (buttons[index]->pressEvent ())
    {
        input.pressedMask |= bit;
    }

    if (buttons[index]->releaseEvent ())
    {
        input.releasedMask |= bit;
    }
}
```

```
input.startEvent = startRequested;
const adk::Status status = game.update (now, input);
```

Initialize a fresh, zeroed `SimonInput` every update. Sample all four buttons before calling `game.update()`. Bits 0–3 map to `CueId::One`–`Four`. A pressed bit without exactly one active bit is invalid during `AwaitPress`; simultaneous buttons therefore fail independently of iteration order. Bits 4–7 are invalid in every phase.

6 State machine



Phase	Accepted transition	Snapshot evidence
Idle	`startEvent`	sequence length already generated
PlaybackOn	cue-on duration due	one `ledMask` bit; displayed cue valid
PlaybackGap	gap due	next cue or `AwaitPress`
AwaitPress	one press, mismatch, chord, or timeout	expected cue and deadline valid
AwaitRelease	`activeMask == 0`	press held until complete release
RoundSuccess	result duration due	source grows sequence, then playback
GameSuccess / Failure	`startEvent`	source reset; same sequence repeats

Boundary rule. At a deadline, the deadline transition wins according to the implementation's branch order. In `AwaitPress`, timeout is checked before input. Test a press exactly at the timeout and document the expected `Timeout`.

7 Golden replay with a fixed source

Use the configuration and source from the API listing. Each row is one call; do not invent intermediate ticks.

ms	input change	phase after update	required snapshot fact
0	start event	PlaybackOn	displayed One; LED mask `0x01`
99	none	PlaybackOn	deadline not due
100	none	PlaybackGap	LED mask zero
150	none	AwaitPress	expected One; input accepted
200	One pressed/active	AwaitRelease	observed One; player index 0
220	One released; active zero	RoundSuccess	`RoundComplete`
320	none	PlaybackOn	length 2; displayed One
420	none	PlaybackGap	LED mask zero
470	none	PlaybackOn	displayed Three; mask `0x04`
570	none	PlaybackGap	LED mask zero
620	none	AwaitPress	expected One
700	One pressed/active	AwaitRelease	observed One
720	One released; active zero	AwaitPress	expected Three
800	Three pressed/active	AwaitRelease	observed Three
820	Three released; active zero	GameSuccess	`GameComplete`

Replay proof. Construct two new sources and games. Feed both the same rows. After every update compare status, phase, outcome, all cue presence flags and values, deadline presence/value, LED mask, sequence length, indices, and input-accepted flag. Final outcome alone is not enough.

7.1 Versioned PRNG

`XorShift32CueSource` reports `SimonAlgorithm::XorShift32V1`. With seed `0x12345678`, the first eight internal states are:

draw	1	2	3	4	5	6	7	8
state (hex)	87985aa5	155b24a3	4820f4c4	81b3ac98	703a0788	29a8e24d	89ca4f1d	c5186e29
cue index	1	3	0	0	0	1	1	1

The cue is the low two state bits, mapped 0–3 to One–Four. Verify the vector exactly. Seed zero is normalized to the documented nonzero seed `0x6d2b79f5`; record the value returned by `seed()`, not merely the requested value. This generator supports replay, not security or gambling.

8 Correctness and fault matrix

Case	Required assertion	Result
fixed golden replay	every snapshot field matches every row	☐
same PRNG seed twice	identical cues and snapshots	☐
different seeds	recorded, deterministic vectors; difference not assumed at every draw	☐
zero seed	normalized seed and vector are stable	☐
press mismatch	`GameFailure/Mismatch`; observed cue retained	☐
two simultaneous presses	`InvalidInput`; no order dependence	☐
input bit 4	update returns `InvalidArgument`	☐
press at timeout	`Timeout` wins	☐
held correct button	remains `AwaitRelease`	☐
release then next press	advances exactly once	☐
fixed source exhausted	`SourceFailure`; status retained	☐
invalid cue from source	rejected deterministically	☐
maximum length 32	capacity succeeds; 33 configuration rejected	☐
timestamp rollover	elapsed deadlines remain correct	☐
backward/ambiguous time	update rejects interval beyond half range	☐
restart after terminal	source reset reproduces sequence	☐
hardware claim failure	reverse cleanup; all outputs inactive/high impedance	☐
shutdown in every phase	tone stopped; PWM zero; pins released	☐

Exercise. In `AwaitPress`, set `pressedMask = activeMask = 0x03`. Predict status, phase, outcome, and whether observed cue is present: _____.

Exercise. A custom source succeeds once and then returns `error() == StatusCode::HardwareFailure`. Identify the phase that discovers this for a starting length of one: _____. Identify the final outcome: _____.

9 Output policy

Apply a snapshot only after a successful engine update:

- cue LEDs follow bits 0–3 of `ledMask`;
- RGB blue means playback, amber means awaiting input, green means success, red means failure, and off means idle;
- sound frequency maps by cue: 262, 330, 392, and 523 Hz;
- cue sound starts only while `hasDisplayedCue`; success/failure sounds are bounded by explicit durations;
- on any adapter error, stop sound, set semantic outputs inactive, then shut down endpoints to high impedance in reverse initialization order.

The engine does not encode colors or musical notes. `CueId` is neutral, so another adapter can use vibration, a display, larger protected drivers, or accessible multimodal feedback without changing game correctness.

10 Mega 2560 acceptance

1. Record repository commit, Arduino CLI and AVR core versions, board ID, supply, meter, and logic-analyzer model if used.
2. Disconnect power. Build one branch at a time from the exact schematics. Verify seven 330Ω resistors,

RGB common cathode, four button contact pairs, passive piezo, and common ground.
3. Compile the supported Simon engine and adapter example for `arduino:avr:mega`. Record flash and static RAM separately.
4. Initialize with the fixed sequence One, Three. Verify all outputs are inactive before successful initialization and after shutdown.
5. Verify each button's raw and debounced diagnostic state separately. Then verify cue-to-button mapping One through Four.
6. Play the golden two-cue game. Observe cue LED, RGB state, tone, and serial snapshot at every transition.
7. Press two buttons as nearly simultaneously as practical. Verify a deterministic invalid-input failure; use a host replay for exact simultaneity.
8. Verify mismatch, timeout, round success, game success, terminal restart, and a button held through reset.
9. Run the same recorded seed twice. Compare the displayed sequence, not just the final score.
10. Invoke shutdown during PlaybackOn, AwaitPress, and sound. Measure cue, RGB, and piezo pins inactive/high impedance. Remove power before teardown.

Field	Record	Field	Record
Commit		Mega board ID	
CLI / core		Supply	
Flash / RAM		Meter / analyzer	
Algorithm		Normalized seed	
Worst LED current		Shutdown measurements	
Seven-die fault bound		Piezo part / rating	

Status language. Until this checklist is signed, report "Simon engine host verified; hardware adapter experimental." Compilation alone is not hardware verification.

11 Diagnose from evidence

Observation	Likely cause	First safe check
one button always active	switch rotated, pin grounded, polarity wrong	power off; continuity test
one press becomes several	raw sample used as event, adapter mask not cleared	replay bounce trace
wrong cue accepted	bit-to-cue map mismatch	fixed One–Four mapping test
two presses depend on order	game updated per button instead of per frame	inspect mask construction
sequence changes on replay	hidden seed/time or source not reset	log version, normalized seed, inputs
cue stays lit in gap	stale snapshot or wrong phase mapping	compare `ledMask`
tone never stops	sounder update omitted or stale command	inspect bounded duration and shutdown
RGB color is inverted	common-anode part or duty inversion	disconnect; verify datasheet
board resets	short or excess aggregate current	disconnect immediately

12 Analysis and extensions

1. Explain why `releasedMask` is recorded even though the current `AwaitRelease` transition uses `activeMask == 0`.
2. Change only timestamps so the golden replay crosses `0xffffffff`. Show each unsigned elapsed calculation.
3. Add difficulty profiles by constructing a new immutable `SimonConfig`; do not mutate live timing midway through a phase.
4. Design a color-blind mode using position, blink count, and distinct tones. RGB color alone must never carry required state.
5. Add a serial replay recorder. Define a terse, versioned line containing time, masks, start event, phase, outcome, LED mask, indices, algorithm, and seed.
6. Replace the four direct LEDs with a shift register in a later lesson. Keep `SimonSnapshot` unchanged and isolate the new timing in the output adapter.

13 Claim, evidence, reasoning

Claim. The same recorded inputs produce the same complete Simon behavior, and the Mega adapter gives each phase unambiguous multimodal feedback.

Evidence. Cite the golden replay, PRNG vector, chord and timeout tests, current calculation, physical mapping trial, and shutdown measurements.

Reasoning. Connect immutable configuration, explicit timestamps, complete masks, source reset, snapshot-driven outputs, and RAII cleanup to the claim. Separate host evidence from hardware observation.

---

14 Further reading

- ADK reference, examples, tests, and complementary HTML: project documentation
- Arduino Mega 2560 Rev3 documentation and schematic: official hardware page
- Microchip ATmega2560 electrical and port limits: official device page
- Arduino tone and GPIO language references: official language reference
- C++ Core Guidelines resource management: resource section
- Xorshift generator family, George Marsaglia, 2003: Journal of Statistical Software

Final sign-off
Wiring checked unpowered: ☐ Golden replay passed twice: ☐ Chord rejected: ☐ Current calculated: ☐
Shutdown measured: ☐ Power removed: ☐